



Gas & Air Quality Sensors (MQ-2, MQ-3, MQ-135)

Hardware Overview



What are they?

The **MQ series** are chemical sensors designed to detect specific types of gases and airborne pollution. They all share the same hardware design and are grouped together in one unified library.

- **MQ-2 (Smoke & Flammables):** Detects smoke, butane, and propane. Build an alarm.
- **MQ-3 (Alcohol Vapor):** Detects ethanol and alcohol vapors. Build a "breathalyzer".
- **MQ-135 (Air Quality):** Detects general Volatile Organic Compounds (VOCs) and stale breath. It answers the simple question: *'Is the air fresh, or do we need to open a window?'*



Electrical Explanation

Inside the metal cap of a MQ sensor is a small heating element and a chemical coating. When target gases hit the hot coating, the electrical resistance of the sensor drops.

These sensors must get HOT to work properly. When you first plug them in, the readings will drift until the heater reaches operating temperature (up to 200 °C).

Key Hardware Facts

- **Connection:** Analog Pin (e.g., A3).
- **Voltage:** 5V (consumes significant power (~800mW) for heating)
- **Signal Flow:** Polluted Air → Chemical Layer → Resistance Drop → ADC → Processor.

Software & Programming



kcomp_MQsensors.h

This library handles the math required to read these sensors and converts the data into a simple, universal 0 to 100% Intensity Score.

Usage: `#include <kcomp_MQsensors.h>`

Function Reference

Function Name	Description & Parameters
<code>void initMQ(uint8_t pin, uint8_t sensor_type)</code>	Setup: Prepares the pin and loads the math for the sensor. ▪ <code>pin</code>: Analog pin the sensor is connected to (e.g., A3). ▪ <code>sensor_type</code>: Use <code>SENSOR_MQ2</code>, <code>SENSOR_MQ3</code>, or <code>SENSOR_MQ135</code>.
<code>void tuneSensitivity(uint8_t pin, float ratio)</code>	Optional Tuning: Changes how easily the sensor triggers. ▪ <code>pin</code>: Analog pin the sensor is connected to (e.g., A3). ▪ <code>ratio</code>: A low number (e.g., 2.0) means high sensitivity; a high number (e.g., 10.0) means low sensitivity.
<code>void calibrateMQ(uint8_t pin)</code>	Calibration: Measures the air to define the 0% baseline. ⚠ <i>Must be run in clean, fresh air!</i> ▪ <code>pin</code>: Analog pin the sensor is connected to (e.g., A3).

<code>uint8_t readSensorScore(uint8_t pin)</code>	Read Data: Returns a percentage from 0% (perfectly clean) to 100% (heavily saturated).
<code>uint8_t readSensorStatus(uint8_t pin)</code>	Status: Returns a simple category perfect for warning LEDs: ▪ STATUS_CLEAN (0-33%) ▪ STATUS_MODERATE (34-66%) ▪ STATUS_HIGH (67-100%)

⚠ Pitfalls & Troubleshooting

- "The Numbers are erratic!"
 - **The Cause:** The heater needs time to stabilize.
 - **The Fix:** Leave the sensor on for at least **20 minutes** before trusting the data.
- "It says the air is perfect (0%) even when it is definitely not perfect!"
 - **The Cause:** Maybe the sensor was calibrated in bad air, making "bad air" the new "zero."
 - **The Fix:** Take the board outside, calibrate the sensor in fresh air.

📄 Sample Program

This universal program initializes the sensor, waits for a button press to calibrate it in fresh air, and prints the 0-100% score to the Serial Monitor.

```
#include <kcomp_MQSensors.h>
#include <kcomp_button_start.h>

// Choose your sensor type here: SENSOR_MQ2, SENSOR_MQ3, or SENSOR_MQ135
const uint8_t ACTIVE_SENSOR = SENSOR_MQ135;
#define MQ_PIN A3

void onStartClicked() {
    Serial.println("\n[!] CALIBRATION STARTED...");
    calibrateMQ(MQ_PIN);
    Serial.println("    Baseline Reset to 0% (Fresh Air).");
}

void setup() {
    Serial.begin(115200);

    initMQ(MQ_PIN, ACTIVE_SENSOR);
    initButtonStart();

    // Optional: Make the sensor more reactive
    // tuneSensitivity(MQ_PIN, 2.5);

    // Start the calibration process when the start button was clicked
    attachButtonStartClick(onStartClicked);

    Serial.println("Press the START button in FRESH AIR to calibrate!");
}

void loop() {
    pollStartButton();

    // Print every second
    static unsigned long lastPrint = 0;
    if (millis() - lastPrint > 1000) {
        lastPrint = millis();
    }
}
```

```
uint8_t score = readSensorScore(MQ_PIN);
uint8_t status = readSensorStatus(MQ_PIN);

Serial.print("Intensity Score: ");
Serial.print(score);
Serial.print("% | Status: ");

if (status == STATUS_CLEAN) Serial.println("CLEAN");
if (status == STATUS_MODERATE) Serial.println("MODERATE");
if (status == STATUS_HIGH) Serial.println("HIGH WARNING");
}
}
```

💡 **Try it out:** You can find the full sample program **SimpleMQGasSensors** in the K-Comp examples folder!